

Practical 2

June 18, 2026

E/22/194

EM2050 Computational Methods

Practical 2

Lagrange (Barycentric) Interpolation of Air Quality Dat

1.

Since $n + 1 = 5$,

$$n = 4$$

The Chebyshev extrema over an interval $[a, b]$ are given by:

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{k\pi}{n}\right), \quad k = 0, 1, 2, \dots, n$$

Here, $a = 0$, $b = 11$, and $n = 4$.

Therefore:

$$x_k = 5.5 + 5.5 \cos\left(\frac{k\pi}{4}\right)$$

Calculating for $k = 0, 1, 2, 3, 4$ and sorting them in increasing order gives the extrema.

```
[1]: import numpy as np

# Define parameters
a = 0
b = 11
n = 4
k = np.arange(n + 1)

# Calculate Chebyshev extrema
x_cheb = 0.5 * (a + b) + 0.5 * (b - a) * np.cos(k * np.pi / n)
x_cheb_sorted = np.sort(x_cheb)

print("Chebyshev extrema in increasing order:")
for val in x_cheb_sorted:
```

```
print(f"{val:.4f}")
```

Chebyshev extrema in increasing order:

```
0.0000
1.6109
5.5000
9.3891
11.0000
```

2.

The measured data points in the table are taken at integer hourly intervals ($t = 0, 1, 2, \dots, 11$).

To find the closest available measured data point for each Chebyshev extremum, we round each calculated value to the nearest whole integer.

```
[2]: # Round the Chebyshev extrema to the nearest integer to find closest available
      ↪ times
t_nodes = np.round(x_cheb_sorted).astype(int)

print(f"Exact extrema: {[round(x, 4) for x in x_cheb_sorted]}")
print(f"Closest available time points (t_nodes): {t_nodes.tolist()}")
```

```
Exact extrema: [np.float64(0.0), np.float64(1.6109), np.float64(5.5),
np.float64(9.3891), np.float64(11.0)]
Closest available time points (t_nodes): [0, 2, 6, 9, 11]
```

3.

Using the selected time points t_k from the previous step, we look up their corresponding PM2.5 concentrations $f(t)$ from the provided dataset to form our interpolation nodes (t_k, y_k) .

```
[3]: # Original data from the assignment table
t_data = np.arange(12)
f_t_data = np.array([18, 20, 19, 23, 27, 30, 28, 25, 24, 22, 20, 19])

# Extract the corresponding PM2.5 values
y_nodes = f_t_data[t_nodes]

print("Selected Interpolation Nodes (t, y):")
for t, y in zip(t_nodes, y_nodes):
    print(f"Hour: {t:<2} | PM2.5: {y}")
```

Selected Interpolation Nodes (t, y):

```
Hour: 0 | PM2.5: 18
Hour: 2 | PM2.5: 19
Hour: 6 | PM2.5: 28
Hour: 9 | PM2.5: 22
Hour: 11 | PM2.5: 19
```

4.

The barycentric weights w_j is:

$$w_j = \prod_{i=0, i \neq j}^n \frac{1}{x_j - x_i}$$

By using the selected time nodes $[0, 2, 6, 9, 11]$:

$$w_0 = \frac{1}{(0-2)(0-6)(0-9)(0-11)}$$

$$w_0 = \frac{1}{(-2)(-6)(-9)(-11)}$$

$$w_0 = \frac{1}{1188}$$

The remaining weights (w_1 through w_4) are below.

```
[4]: def compute_barycentric_weights(x_nodes):
    n = len(x_nodes)
    weights = np.zeros(n)
    for j in range(n):
        # Calculate product of (x_j - x_i) for all i != j
        diffs = x_nodes[j] - np.delete(x_nodes, j)
        weights[j] = 1.0 / np.prod(diffs)
    return weights

weights = compute_barycentric_weights(t_nodes)

print("Calculated Barycentric Weights:")
for i, w in enumerate(weights):
    print(f"w_{i} = {w:.6f}")
```

Calculated Barycentric Weights:

```
w_0 = 0.000842
w_1 = -0.001984
w_2 = 0.002778
w_3 = -0.002646
w_4 = 0.001010
```

5.

The second form of the barycentric interpolation formula is:

$$P(x) = \frac{\sum_{j=0}^n \frac{w_j}{x-x_j} y_j}{\sum_{j=0}^n \frac{w_j}{x-x_j}}$$

Evaluate this at $t = 4.5$ using the nodes and weights calculated in the previous steps.

```
[5]: def barycentric_interpolate(x_eval, x_nodes, y_nodes, weights):
    # Standard barycentric formula
    terms = weights / (x_eval - x_nodes)
    numerator = np.sum(terms * y_nodes)
    denominator = np.sum(terms)
    return numerator / denominator

t_eval = 4.5
pm25_estimate = barycentric_interpolate(t_eval, t_nodes, y_nodes, weights)

print(f"Estimated PM2.5 concentration at t = {t_eval} is: {pm25_estimate:.4f} µg/
    ↳m³")
```

Estimated PM2.5 concentration at t = 4.5 is: 26.4484 µg/m³

6.

To find 5 equally spaced nodes over the interval [0, 11]:

$$\text{Spacing} = \frac{11 - 0}{4} = 2.75$$

The exact equally spaced points are: 0.00, 2.75, 5.50, 8.25, 11.00.

Rounding to the closest available hourly measurements gives [0, 3, 6, 8, 11].

Equally spaced measured nodes:

$$[0, 3, 6, 8, 11]$$

Chebyshev like measured nodes:

$$[0, 2, 6, 9, 11]$$

The Chebyshev like nodes are clustered closer to the boundaries of the interval. The equally spaced nodes are distributed more uniformly. The Chebyshev clustering helps minimize Runge's phenomenon at the edges of the interpolation polynomial.

```
[6]: import matplotlib.pyplot as plt

# The nodes we calculated
t_cheb = [0, 2, 6, 9, 11]
t_eq = [0, 3, 6, 8, 11]

# Create a 1D number line comparison plot
plt.figure(figsize=(10, 3))

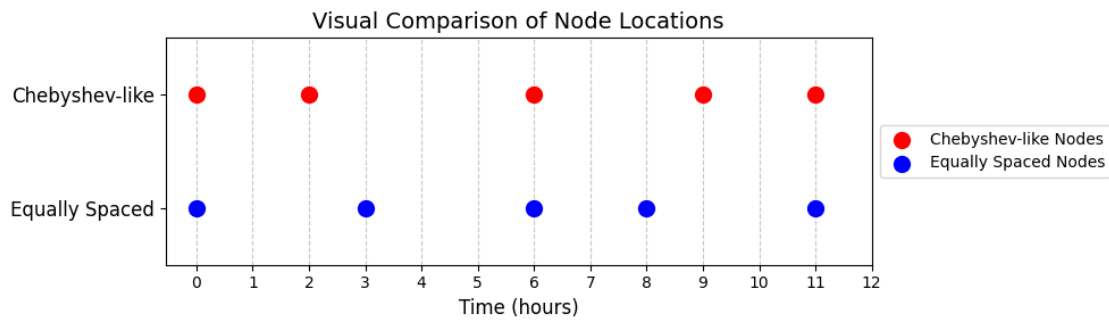
# Plot the nodes
plt.scatter(t_cheb, [1]*len(t_cheb), color='red', s=100, label='Chebyshev-like
    ↳Nodes', zorder=5)
plt.scatter(t_eq, [0]*len(t_eq), color='blue', s=100, label='Equally Spaced
    ↳Nodes', zorder=5)
```

```

# Formatting the plot
plt.yticks([0, 1], ['Equally Spaced', 'Chebyshev-like'], fontsize=12)
plt.xticks(range(13))
plt.xlabel('Time (hours)', fontsize=12)
plt.title('Visual Comparison of Node Locations', fontsize=14)
plt.grid(True, axis='x', linestyle='--', alpha=0.7)
plt.ylim(-0.5, 1.5)
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))

plt.tight_layout()
plt.show()

```



Tast1:Understanding the Node Selection Procedure

1.

The function $\text{fx}(a,b,n)$ generates the **Chebyshev extrema** mapped over an arbitrary interval $[a, b]$.

The corresponding mathematical formula is:

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{k\pi}{n}\right), \quad \text{for } k = 0, 1, 2, \dots, n$$

2.

Because they significantly minimize Runge's phenomenon.

When using equally spaced nodes for high degree polynomial interpolation the resulting polynomial tends to oscillate wildly near the edges of the interval. Chebyshev nodes solve this by clustering the interpolation points more densely near the boundaries . This distribution yields an interpolating polynomial that is exceptionally stable and provides a near optimal uniform approximation of the underlying function.

3.

1. **Distance Calculation:** It computes the absolute difference (`np.abs`) between every ideal Chebyshev point and every available time point in the `domain`.
2. **Finding the Minimum:** It uses `np.argmin` to find the index of the available data point that has the smallest distance to each ideal Chebyshev point.
3. **Removing Duplicates:** It applies `np.unique` to the selected indices. This ensures that if two ideal Chebyshev points happen to snap to the exact same available data point, the duplicate is removed, leaving only a set of strictly unique interpolation nodes.

4.

The overall purpose of this code is instead of interpolating through all available data points, or just picking random/evenly spaced subset of points, the script intentionally selects a smaller subset of points that closely mimic the optimal Chebyshev distribution. It extracts the raw data at these strategically chosen times, builds a barycentric interpolation polynomial using only those nodes, and then evaluates that smooth curve over the entire time domain for visualization.

Task2: Large-Scale Interpolation of Air Quality Data

```
[8]: import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import BarycentricInterpolator
import pandas as pd
import io
import os

# Import Colab's interactive file upload library
try:
    from google.colab import files
except ImportError:
    print("Warning: This upload widget only works inside Google Colab.")

# =====
# 0. Interactive File Upload & Load Data
# =====
print("Please upload your air quality dataset ('airdata.npy' or an Excel file):")
uploaded = files.upload()

# Get the name of the file just uploaded
filename = list(uploaded.keys())[0]

if filename.endswith('.npy'):
    data = np.load(filename)
    print(f"\nSuccessfully loaded '{filename}'!")
elif filename.endswith(('.xlsx', '.xls')):
```

```

# Load Excel file directly from the uploaded bytes
df = pd.read_excel(io.BytesIO(uploaded[filename]))
# Assuming PM2.5 is the first column. Change df.iloc[:, 0] to
→df['YourColumnName'] if needed.
data = df.iloc[:, 0].to_numpy()
print(f"\nSuccessfully loaded '{filename}'!")
else:
    raise ValueError("Unsupported file format. Please upload a .npz or Excel
→file.")

# Set up domain
N = len(data)
a = 0
b = (N - 1) / 24 # Domain in days (assuming hourly data)
domain = np.linspace(a, b, N)

# =====
# 1-5. Interpolation & Plotting Function
# =====
def plot_barycentric_interpolation(n, use_chebyshev=True):
    """
    Approximates data using barycentric interpolation and plots the result.
    """

    # 1. Generate ideal extrema over [a, b]
    if use_chebyshev:
        k = np.arange(n + 1)
        ideal_points = 0.5 * (a + b) + 0.5 * (b - a) * np.cos(k * np.pi / n)
        node_type = "Chebyshev-like"
    else:
        ideal_points = np.linspace(a, b, n + 1)
        node_type = "Equally Spaced"

    ideal_points = np.sort(ideal_points)

    # 2. Find the closest available data point
    temp = np.abs(ideal_points - domain.reshape(N, 1))
    idx = np.argmin(temp, axis=0)
    idx = np.unique(idx) # Remove duplicates

    xnodes = domain[idx]
    ynodes = data[idx]

    # 3. Construct barycentric interpolating polynomial
    poly = BarycentricInterpolator(xnodes, ynodes)
    yinterp = poly(domain)

```

```

# 4 & 5. Plotting (Two Subplots)
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 10))
fig.suptitle(f'Barycentric Interpolation using ~{n+1} {node_type} Nodes',
fontsize=16)

# --- Subplot 1: Full Data Range ---
ax1.plot(domain, data, color='lightgray', label="Original Data", alpha=0.7)
ax1.plot(domain, yinterp, color='blue', linewidth=1.5, label="Interpolating
Polynomial")
ax1.scatter(xnodes, ynodes, color='red', s=30, zorder=5, label="Selected
Nodes")
ax1.set_title("Full Domain Interpolation")
ax1.set_xlabel("Time (days)")
ax1.set_ylabel("PM2.5 Concentration")
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.6)

# --- Subplot 2: Zoomed in for clarity (First 30 days) ---
zoom_days = 30
mask = domain <= zoom_days

ax2.plot(domain[mask], data[mask], color='lightgray', label="Original Data",
alpha=0.7)
ax2.plot(domain[mask], yinterp[mask], color='blue', linewidth=1.5,
label="Interpolating Polynomial")

# Filter nodes for the zoomed plot
node_mask = xnodes <= zoom_days
ax2.scatter(xnodes[node_mask], ynodes[node_mask], color='red', s=40,
zorder=5, label="Selected Nodes")

ax2.set_title(f"Zoomed View (First {zoom_days} Days) - Shows Approximation
Clearly")
ax2.set_xlabel("Time (days)")
ax2.set_ylabel("PM2.5 Concentration")
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.6)

# Y-axis limits to handle extreme Runge's phenomenon explosions
ymin, ymax = np.min(data[mask]), np.max(data[mask])
padding = (ymax - ymin) * 0.5
ax2.set_ylim(ymin - padding, ymax + padding)

plt.tight_layout()
plt.show()

```



```
# =====
# 6. Execute Code for n = 50
# =====

n_intervals = 50

# Run with Chebyshev-like nodes
print("\nExecuting with Chebyshev-like nodes...")
plot_barycentric_interpolation(n_intervals, use_chebyshev=True)

# Run with Equally Spaced nodes
print("\nExecuting with Equally Spaced nodes...")
plot_barycentric_interpolation(n_intervals, use_chebyshev=False)
```

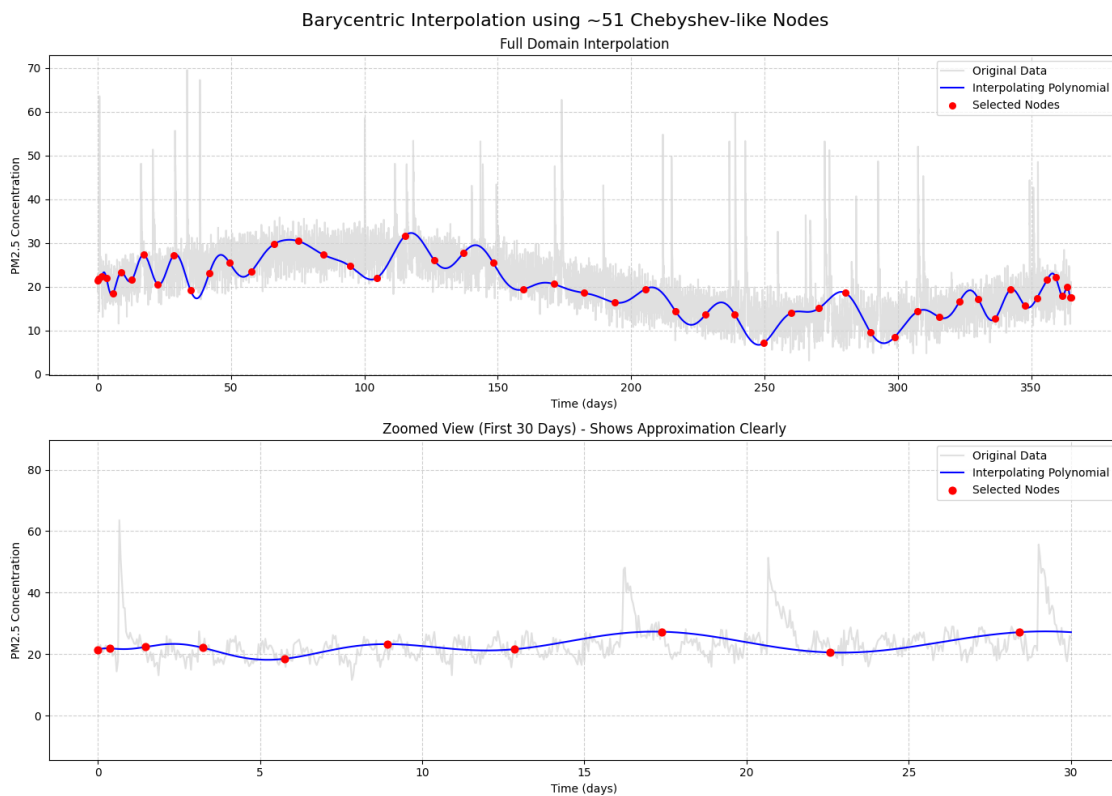
Please upload your air quality dataset ('airdata.npy' or an Excel file):

<IPython.core.display.HTML object>

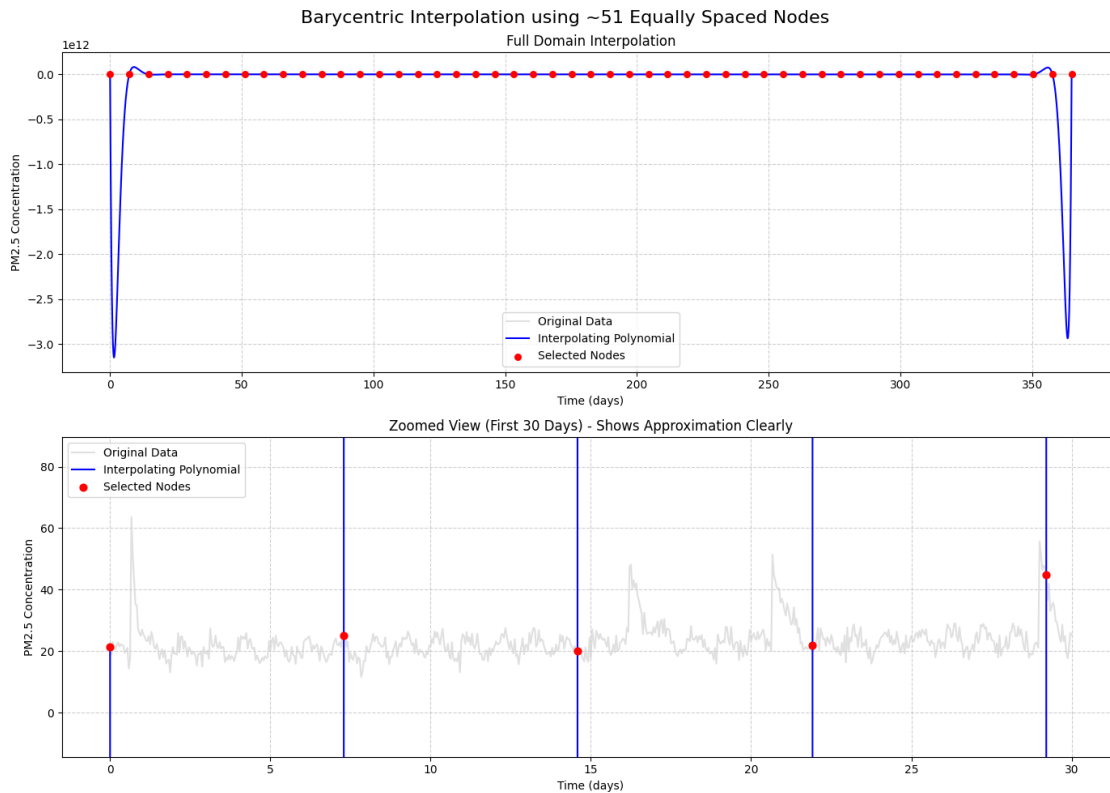
Saving airdata.npy to airdata.npy

Successfully loaded 'airdata.npy'!

Executing with Chebyshev-like nodes...



Executing with Equally Spaced nodes...



7.

Based on the generated plots, the difference between using equally spaced nodes and Chebyshev-like nodes is extreme.

- **Interpolation accuracy:** The Chebyshev like nodes provide a highly accurate, smooth, and uniform approximation of the PM2.5 data across the entire year. The error is so massive that the polynomial fails to represent the data at all.
- **Behaviour near the endpoints:** The plots perfectly illustrate Runge's phenomenon. In the equally spaced plot, the interpolating polynomial explodes near the endpoints, oscillating wildly and reaching physically impossible values on the order of 10^{12} . The Chebyshev nodes intentionally cluster near these boundaries, which effectively suppresses these oscillations and keeps the polynomial tightly bound to the actual data range.
- **Numerical stability:** Barycentric interpolation with Chebyshev nodes is highly stable (forward-stable). However, with equally spaced nodes at a high degree ($n = 50$), the method becomes numerically unstable. The barycentric weights for equally spaced points grow exponentially and alternate in sign, which causes catastrophic cancellation errors in floating-point arithmetic, contributing to the massive vertical spikes seen in the zoomed in plot.
- **Computational cost:** The algorithmic computational cost is identical for both methods.

Constructing the interpolant takes the same number of operations ($\mathcal{O}(n^2)$ to compute weights, or $\mathcal{O}(n)$ if using analytical formulas), and evaluating it takes $\mathcal{O}(n)$ per point. Therefore, choosing Chebyshev nodes provides a massive upgrade in accuracy and stability without any extra computational penalty.